

Arrays and vectors

CSC 240 Data Structures Fall 2025

Table of Contents

- [1. Objectives](#)
- [2. Arrays \(analogy\)](#)
- [3. C-style arrays as data structures \(top-down\)](#)
- [4. Simple example: Array declaration \(bottom-up\)](#)
- [5. Two types of initialization](#)
- [6. PRACTICE Extended example: Declare and Display Product Prices](#)
- [7. Inputting array content element-wise](#)
- [8. Stepping through an array](#)
- [9. PRACTICE Running C++ on the command-line \(CLI\)](#)
- [10. Initializing a string array](#)
- [11. Partial array initialization](#)
- [12. The Range-based for loop](#)
- [13. PRACTICE Range-based for Loop temperature conversion](#)
- [14. Modifying an array with a Range-Based for Loop](#)
- [15. PRACTICE Range-based for loop \(+ command-line & make\)](#)
- [16. PRACTICE Store and display strings using range-based loops](#)
- [17. Processing array contents](#)
- [18. Assigning arrays](#)
- [19. Summing the Values in a Numeric Array](#)
- [20. \[Bonus program\] Getting the Average of the Values in a Numeric Array](#)
- [21. \[Bonus program\] Finding the Highest and Lowest Values in a Numeric Array](#)
- [22. Partially Filled Arrays and Sentinels](#)
- [23. \[Bonus program\] Fill array partially with sentinel control](#)
- [24. Comparing arrays](#)
- [25. Primer on functions](#)
- [26. Arrays as Function Arguments](#)
- [27. NEXT Introduction to the STL vector](#)
- [28. Declaring vectors](#)
- [29. PRACTICE Storing values in a vector: Payroll program -> CANVAS](#)
- [30. Scope](#)
- [31. Using the range-based for loop with a vector \(from C++11\)](#)
- [32. PRACTICE Using the push_back member function \(payroll pgm modified\) -> CANVAS](#)
- [33. Determining the size of a vector](#)
- [34. Removing elements from a vector with pop.back](#)
- [35. Clearing a vector](#)
- [36. Detecting an empty vector](#)
- [37. PRACTICE Summary of vector member functions](#)

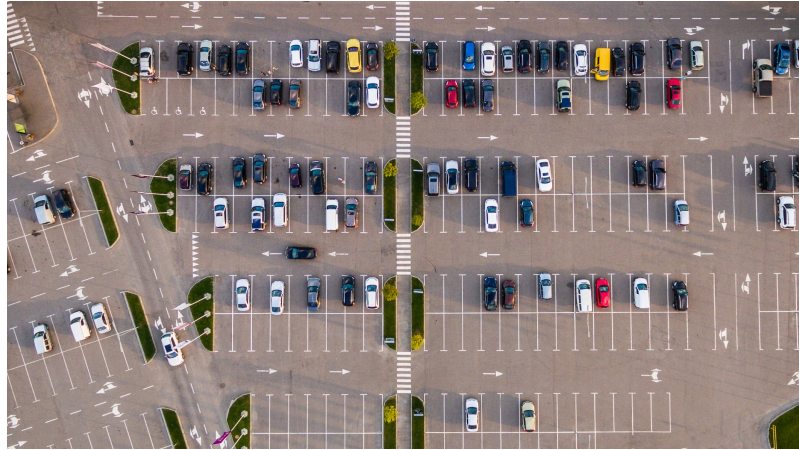
1. Objectives

- [] Top-down/Analogy/Bottom-up explanation of arrays
- [] Array declaration and initialization
- [] Array input without loops
- [] Stepping through an array with for

- [] Running C++ on the command-line with make
- [] Range-based for loops
- [] Assigning and comparing arrays
- [] Partially filled arrays
- [] Sentinel controlled arrays
- [] Arrays as function arguments

2. Arrays (analogy)

- An array is a little like a parking lot:



1. Each parking space is an **element**.
2. The space number is the **index**.
3. The vehicle type allowed is the **type**.
4. The parking lot has a fixed **size**.

- An array is even more like a train:



1. Each train car is an **element**.
2. Cars are coupled permanently, or **contiguously**.
3. The car number is the **index**.
4. Trains have a **type**: passenger or cargo.
5. You can move from one car to the next like a **pointer**.
6. Removing one car in the middle is difficult.

3. C-style arrays as data structures (top-down)

- What do we want to know about a data structure when we first encounter it? (Think "array" if you like.)
 1. What is its purpose?
 2. How are the data stored?
 3. How do you define it?
 4. How do you access it?
 5. How do you copy it?
 6. How do you extend it?
 7. How do you delete it?
 8. What else is worth knowing about it?

4. Simple example: Array declaration (bottom-up)

- Complete the following program (let's do this together as a warmup):
 1. Supply the definition for an integer array named `values` of size `NUM_ELEMENTS`, and assign the values 100, 200, 300, 400, 500.
 2. Print the values of the array on one line separated by a space.
- Starter code: <https://tinyurl.com/array-declaration>

```
// Declare, initialize and print an array of 5 elements
#include <iostream>
using namespace std;

int main()
{
    // Define size of array NUM_ELEMENTS

    // Define int array `values` of size NUM_ELEMENTS and assign
    // 100...500 to the array

    // Print the array elements on one line separated by a space
    return 0;
}
```

- Solution: <https://onecompiler.com/cpp/43vgaj8dc>

```
#include <iostream>
using namespace std;

int main()
{
    // Define size of array NUM_ELEMENTS
    const int NUM_ELEMENTS = 5;
    // Define integer array `values` of size NUM_ELEMENTS
    // Assign 100...500 to the array
```

```
int values[NUM_ELEMENTS] {100, 200, 300, 400, 500};

// Print the array elements on one line separated by a space:
// Either: As a stream of indexed elements
cout << values[0] << " " << values[1] << " " << values[2] << " " << values [3] <
// Or: As a loop over the index
for (int i=0; i<NUM_ELEMENTS; i++) cout << values[i] << " ";

return 0;
}
```

5. Two types of initialization

1. Traditional C-style copy syntax: `value[2]={100,200};`
2. C++ direct list initialization: `value[2] {100,200};`

The C++ version is safer: It avoids "narrow conversions", which means e.g. to implicitly turn a float value into a int val

```
double arr[2] = {100,200}; // OK: int->double widening conversion
int arr2[2] {100.,200.}; // ERROR: double->int narrowing
```

6. **PRACTICE** Extended example: Declare and Display Product Prices

You need to upload the URL for this practice file to Canvas.

Scenario

You've joined the IT support team at a small e-commerce startup, QuickCart. Your first task is to write a simple C++ program that stores the fixed prices of five featured products. These prices must be stored in an array and displayed on a single line for the company dashboard preview.

Task

- Complete the program below:
 1. Define an floating-point array named `prices` with size `SIZE`.
 2. Assign the values `10.0 100.0 45.99 30.00 200.00` to the array (these represent product prices in dollars).
 3. Print the array elements on one line separated by one space.
- **Expected Output:**

```
10 100 45.99 30 200
```

Starter Code: tinyurl.com/array-product-prices

```
// Store product prices in an array
// Author:

#include <iostream>
```

```
using namespace std;

int main()
{
    // Set SIZE of array

    // Define array `prices` and assign product prices

    //Print prices on one line

    return 0;
}
```

Tip

- Remember to use brace list initialization for the array.
- You can use a loop to print each element with a trailing space.

Challenges (for home assignment)

1. Let a user enter 5 product prices instead of hardcoding them
2. Validate that each input is a positive number.

Solution in class

```
// Store product prices in an array
// Author:

#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    // Set SIZE of array
    const int SIZE = 5;
    // Define array `prices` and assign product prices
    double prices[SIZE] {10.0,100.0,45.99,30.00,200.00};
    // Set precision
    cout << fixed << setprecision(2);
    // Print prices on one line
    for (int i=0; i<SIZE; i++)
        cout << prices[i] << " ";
    return 0;
}
```

7. Inputting array content element-wise

- **Task:** Write a program that asks for the number of hours worked by six employees to be entered on the command line. Store the values in an array, and then print them on one line separated by one space.
Constraint: **Do not use loops.**
- **Sample input:** 20 12 40 30 30 15
- **Sample output:**

Enter the hours worked by 6 employees:
The hours you entered were: 20 12 40 30 30 15

- **C++ starter code:** tinyurl.com/employee-hours

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Do not use loops at all.
// Author:

// include header file
// use std namespace
// main program
// define array size declarator NUM_EMPLOYEES
// define array `hours`
// ask user for keyboard input
// get array input from keyboard
// print output
```

- **Solution:**

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Do not use loops at all.
// Author: Marcus Birkenkrahe (pledged)
// include header file
#include <iostream>
// use std namespace
using namespace std;
// main program
int main()
{
    // define array size declarator
    const int NUM_EMPLOYEES = 6;
    // define array
    int hours[NUM_EMPLOYEES];
    // ask user for keyboard input
    cout << "Enter the hours worked by "
         << NUM_EMPLOYEES << " employees: ";
    // get array input from keyboard
    cin >> hours[0];
    cin >> hours[1];
    cin >> hours[2];
    cin >> hours[3];
    cin >> hours[4];
    cin >> hours[5];
    // print output
    cout << "\nThe hours you entered were:";
    cout << " " << hours[0];
    cout << " " << hours[1];
    cout << " " << hours[2];
    cout << " " << hours[3];
    cout << " " << hours[4];
    cout << " " << hours[5];

    return 0;
}
```

Input file:

```
echo "20 12 40 30 30 15" > input
cat input
```

```
20 12 40 30 30 15
```

8. Stepping through an array

- Change the previous solution to get the keyboard input and generate the output using a loop to step through the array.
- Solution: tinyurl.com/employee-hours-2

```
// Program asks for number of hours worked by six employees to be
// entered on the command-line. Values are stored in an array and
// printed separated by one space. Use for loops.
// Author: Marcus Birkenkrahe (pledged)
// include header file
#include <iostream>
// use std namespace
using namespace std;
// main program
int main()
{
    // define array size declarator
    const int NUM_EMPLOYEES = 6;
    // define array
    int hours[NUM_EMPLOYEES];
    // ask user for keyboard input
    cout << "Enter the hours worked by "
          << NUM_EMPLOYEES << " employees: ";
    // get array input from keyboard with a loop
    for (int i=0; i<NUM_EMPLOYEES; i++) cin >> hours[i];
    // print output
    cout << "\nThe hours you entered were:";
    for (int i=0; i<NUM_EMPLOYEES; i++) cout << " " << hours[i];
    return 0;
}
```

```
echo "20 12 40 30 30 15" > input
```

9. **PRACTICE** Running C++ on the command-line (CLI)

1. Open the Google Cloud Shell at ide.cloud.google.com
2. Open the terminal
3. Download the source code from the web:

```
wget -O hours.cpp tinyurl.com/step-through-array
```

4. Check the file with `cat hours.cpp`
5. Compile the file with `make hours`
6. Run the file with `./hours` and enter 20 12 40 30 30 15
7. Upload screenshot of your terminal with the result to Canvas.

10. Initializing a string array

- Write a program that **safely** initializes a string array named `planets` with the names of the planets of our solar system (ordered by distance from the Sun). Print the result. Make sure that you can tangle the source code file as `planets.cpp` if needed.
- Sample output:

```
Planets of our solar system:  
Mercury  
Venus  
Earth  
Mars  
Jupiter  
Saturn  
Uranus  
Neptune
```

- Starter code: <https://tinyurl.com/string-array>

```
// Initialize string array `planets` with the names of the 8 planets  
// of our solar system (ordered by distance from the sun). Print it.  
// header files  
// use std namespace  
// main pgm  
// define array SIZE  
// define and initialize string array planets  
// print the names of the planets of our solar system
```

11. Partial array initialization

- You can partially initialize arrays. Example:

```
const int SIZE = 7;  
int num[SIZE] {1,2,3,4};  
for (int i=0; i<SIZE; i++) cout << num[i] << " ";
```

```
1 2 3 4 0 0 0
```

- You can implicitly size arrays if you initialize them with a list. Example:

```
double ratings[] {1.3, 6.6, 8.4, 2.2};  
cout << sizeof(ratings)/sizeof(ratings[0]);
```

```
4
```

- An alternative size check uses `std::size` from `<iterator>`:

```
#include <iterator>  
double ratings[] {1.3, 6.6, 8.4, 2.2};
```



```
cout << std::size(ratings);
```

```
4
```

12. The Range-based for loop

- This loop iterates once for each element in an array. It was introduced in C++11.
- General form:

```
for (dataType rangeVariable : array)  
    statement
```

- Here, `dataType` is the data type of the `rangeVariable`, which receives a different array element during each loop iteration.
- `array` is the array the loop operates on, and `statement` is what you want to do in an iteration.
- Example: Use range-based iteration to display the array: 3,6,9.

```
int numbers[] {3,6,9};  
for (int val : numbers) // loop with val over numbers  
    cout << val << " "; // give me each val
```

```
3 6 9
```

- You can use the auto key word to infer the data type of the range variable:

```
int numbers[] {3,6,9};  
for (auto val : numbers) // compiler will identify int  
    cout << val << " ";
```

```
3 6 9
```

- Example: Display the elements of a string array.

```
std::string names[] {"Jim", "Joe", "Jane", "Jack"};  
for(std::string name : names) cout << name << endl;  
for(auto name : names) cout << name << endl;
```

```
Jim  
Joe  
Jane  
Jack  
Jim  
Joe  
Jane  
Jack
```

13. PRACTICE Range-based for Loop temperature conversion

(PRACTICE = upload the URL of your program(s) later to Canvas).

Write a small C++ program that does the following:

1. Define an array of six floating-point temperatures in Celsius: 22.5, 25.0, 19.5, 21.0, 23.5, 26.1.
2. Use a range-based for loop with auto to:
 - Convert each temperature to Fahrenheit using the formula: $F = C * 9/5 + 32$
 - Print each converted temperature on a new line.
 - Sample output line: "22.5C = 72.5F"

Bonus: Use `std::setprecision` from `<iomanip>` to format the output to one decimal precision.

- Solution: Code along in onecompiler.com/cpp

```
double temps[] {22.5, 25, 19.5, 21.0, 23.5, 26.1};
for (auto temperature : temps) // loop over the array
    cout << temperature << "C = "
        << temperature * 9./5. + 32. << "F." << endl;
```

```
22.5C = 72.5F.
25C = 77F.
19.5C = 67.1F.
21C = 69.8F.
23.5C = 74.3F.
26.1C = 78.98F.
```

- Bonus solution:

1. `fixed`: Forces output to use fixed-point notation (not scientific).
2. `showpoint`: Always show a decimal point, even for whole numbers.
3. `setprecision(1)`: Set number of digits after decimal point to one.

```
#include <iostream>
#include <iomanip>
int main()
{
    double temps[] {22.5, 25, 19.5, 21.0, 23.5, 26.1};
    for (double temperature : temps) // loop over the array
        cout << fixed << showpoint << setprecision(2)
            << temperature << "C = "
            << temperature * 9./5. + 32. << "F." << endl;
    return 0;
}
```

```
22.50C = 72.50F.
25.00C = 77.00F.
19.50C = 67.10F.
21.00C = 69.80F.
23.50C = 74.30F.
26.10C = 78.98F.
```

- What C++ enables you to do is work on three different levels:

values	22.5	25.0	19.5	range-based loops
index	0	1	2	index loops
memory	0x1	0x2	0x3	pointer arithmetic

14. Modifying an array with a Range-Based for Loop

- As the range-based loop executes, the range variable receives a copy of an array element. To change the contents of an array, the range variable needs to be declared as a **reference**.
- What is a reference?

A reference in C++ is an **alias** for an existing variable:

1. It must be initialized when it is declared.
2. It cannot be changed to refer to something else.
3. Any operation on the reference affects the original.
4. No extra storage is required.

Syntax: & (address-of-operator)

- Code example:

```
int x = 10; // integer variable
int &ref = x; // ref is a reference variable referencing x
ref = 20;
cout << "x = " << x << endl;
```

```
x = 20
```

- Example:** Declare integer array of SIZE 5, get values for the array from inputFile in the ../data directory, and display the array values. Use a range-based for loop for both iterations.

```
#include <iostream>
#include <fstream> // contains ifstream and ofstream

int main()
{
    // set array SIZE to 5
    const int SIZE {5};
    // declare array `numbers`
    int numbers[SIZE];
    // declare input file
    std::ifstream inputFile;
    // Get values for the array from input file
    inputFile.open("../data/inputfile");

    for (int &val : numbers)
        inputFile >> val; // read data from the filestream

    inputFile.close();
    // Display values in the array
    for (int val : numbers)
        std::cout << val << " "; // stream data to stdout
```

```
    return 0;
}
```

```
101 102 103 104 105
```

- Input file:

```
cd ../data
echo "101 102 103 104 105" > inputfile
cat inputfile
```

```
101 102 103 104 105
```

- Why does the first loop use `&val` while the second loop uses `val`?

The first loop needs to change the values of the original array numbers, and the reference `&val` provides access to it, while in the second array, we only need a copy of numbers to print it.

- What's the advantage of range-based loops?

Easier to read and safer because there is no out-of-bounds possible.

- What's the disadvantage of range-based loops?

When using range-based loops you do not have access to individual subscripts.

15. **PRACTICE** Range-based for loop (+ command-line & make)

- Starter code: tinyurl.com/range-warmup
- What will be displayed if the user enters the value 5?

```
// Change value of an array using a range-based loop
// Input: 5
// Output: 17
int values[] {12};
for (int value : values)
{
    cout << "Enter a value: ";
    std::cin >> value;
}
cout << '\n' << values[0] << endl;
```

```
Enter a value:
12
```

- Tangle the code: (org-babel-tangle) then build it with make

```
cd ../src                                # change to directory ../src
[ -f Makefile ] && rm Makefile           # if exists, remove Makefile
[ -f values.cpp ] && touch values.cpp     # if exists, touch values.cpp
make values                             # build executable `values`
echo 5 | ./values                        # run `values` with input 5
```

```
g++    values.cpp    -o values
Enter a value:
5
```

- Answer:

The array values will not be modified because the range-based loop works only on a copy of values, not on the original, so value never touches the original array.

The second loop only displays the array elements but does not alter them.

- To change that, loop over the reference variable &value instead: Now value is a reference to the original element of values and modifies it.

```
// Change value of an array using a range-based loop
// Input: 5
// Output: 17
int values[] {12};
for (int &value : values)
{
    cout << "Enter a value: ";
    std::cin >> value;
}
cout << '\n' << values[0] << endl;
```

```
Enter a value:
12
```

- Tangle the code: (org-babel-tangle) - build using make

```
cd ../src                                # change to directory ../src
[ -f Makefile ] && rm Makefile           # if exists, remove Makefile
[ -f values2.cpp ] && touch values2.cpp  # if exists, touch values2.cpp
make values2                             # build executable `values`
echo 5 | ./values2                       # run `values` with input 5
```

```
g++    values2.cpp    -o values2
Enter a value:
5
```

16. **PRACTICE** Store and display strings using range-based loops

- **Task:** Write a C++ program that prompts the user to enter their first, middle, and last names. Store these values in an array of three string objects. Then display the full name.

- **Sample input:**

James Earl Jones

- **Sample output:**

Enter name:
Enter name:
Enter name:
Full name: James Earl Jones

- **Starter code:** The program below does this with a regular for loop. Modify it to work with a range-based loop instead.

Link: <https://tinyurl.com/range-based>

```
// Get first, middle and last user name and print full name.
// TODO: Modify the program to work with range-based loops.
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE {3};    // string array length
    string fullName[SIZE]; // string array declaration

    // get array elements from keyboard
    for (int count=0; count<SIZE; count++) {
        cout << "\nEnter name: ";
        cin >> fullName[count];
    }

    // print result from array
    cout << "\nFull name: ";
    for (int count=0; count<SIZE; count++)
        cout << fullName[count] << " ";

    return 0;
}
```

Enter name:
Enter name:
Enter name:
Full name: 5 12 18

- **Create inputFile:**

```
cd ../data
echo "James Earl Jones" > inputFile
cat inputFile
```

James Earl Jones

- Upload the URL to your solution to Canvas.
- Solution:

```
// Get first, middle and last user name and print full name.
// TODO: Modify the program to work with range-based loops.
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int SIZE {3};    // string array length
    string fullName[SIZE]; // string array declaration

    // get array elements from keyboard
    for (std::string &name : fullName) {
        cout << "\nEnter name: ";
        cin >> name;
    }

    // print result from array
    cout << "\nFull name: ";
    for (std::string name : fullName) {
        cout << name << " ";
    }
    return 0;
}
```

```
Enter name:
Enter name:
Enter name:
Full name: James Earl Jones
```

17. Processing array contents

- Processing array elements is no different than processing other variables. This includes use of pre- and post-increment operations.
- Example: Increase score values in an array

```
int score[] {7,8,9,10,11};
cout << score[2] << endl;
cout << ++score[2] << endl;
cout << score[4]++ << endl;
cout << score[4] << endl;
```

```
9
10
11
12
```

- An easy mistake is to confuse operations on the value with operations on the subscript:

```
count = 1;
value[count--]; // value[1] -> value[0]
value[count]--; // value[1] -> value[1] - 1
```

18. Assigning arrays

- Does the following code work? Why or why not?

```
const int SIZE {4};
int oldValues[SIZE] {10,100,200,300};
int newValues[SIZE];
newValues = oldValues;
```

- Answer:

The only way to assign one C-style array to another is to assign the individual elements in the array.

```
const int SIZE {4};
int oldValues[SIZE] {10,100,200,300};
int newValues[SIZE];

for (int count=0; count<SIZE; count++)
{
    newValues[count] = oldValues[count];
    cout << newValues[count] << " ";
}
```

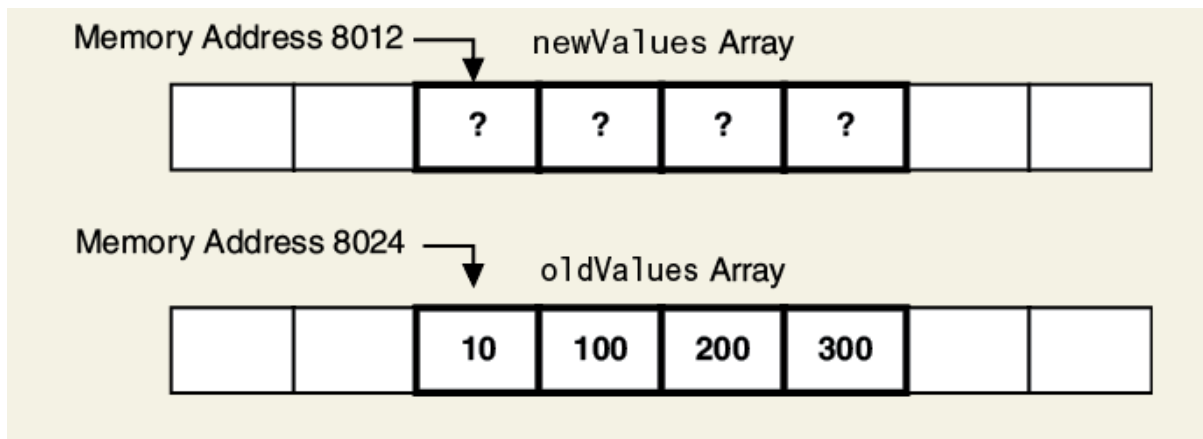
```
10 100 200 300
```

- Explanation:

The reason why the array assignment does not work is that you would have to change the STARTING MEMORY ADDRESS of the array.

Using newValues in code means that the compiler sees: &newValues[0]

newValues = oldValues is seen as 8012 = 8024



- If num is an array of int numbers, what will `cout << num;` print?

```
int num[] {10,20,30,40,50};  
cout << num; // this is the starting address  
cout << '\n' << &num[1]; // one int = 4 bytes forward
```

```
0x7ffc1df9aa20  
0x7ffc1df9aa24
```

- Explanation:

Notice that the address of `num[1]` is 4 bytes = length of an int in memory forward from the address of `num[0]`.

19. Summing the Values in a Numeric Array

- **Task:** Write a short C++ program that sums the values in an array. Can you use a range-based loop? (5 lines)
- **Sample input:** 1,2,3,4,5,6,7,8
- **Sample output:**

```
Sum = 36
```

- **Solution:**

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int arr[] {1,2,3,4,5,6,7,8};  
    int sum = 0;  
    for (int val : arr)  
        sum += val;  
    cout << "Sum = " << sum;
```

```
    return 0;
}
```

```
Sum = 36
```

20. [Bonus program] Getting the Average of the Values in a Numeric Array

- **This will be a bonus program to be completed at home.**
- **Task:** Write a short C++ program that sums the values in an array scores. The first step towards the average is to sum up all the values in the array, and the second is to divide by the length of the array.
- **Sample input:** 10,22,34,42,50,66,71,110
- **Sample output:**

```
Average = 405/8 = 50.625
```

21. [Bonus program] Finding the Highest and Lowest Values in a Numeric Array

- **This will be a bonus program to be completed at home.**
- **Task:** Write a short C++ program that finds the highest value in a numeric array, values, with the values 1,2,3,4,5,6,7,8.
- **Algorithmic Idea:**

To find the highest value, assume that the first value is already highest, then step through the array comparing each element and replacing it with the highest if it is found to be higher.

- It is worth looking at edge cases:
 1. What if all values of the array are identical?
 2. What if the array is empty?
 3. What if some values are negative?
 4. What if the values add up to a very large value?
- Run the code for a few edge cases like very large numbers.

22. Partially Filled Arrays and Sentinels

- What if you want to store items in an array but you don't know how many items there are exactly?

You could make the array very large, large enough for the largest possible number of items.
Not efficient!

- Example: The count variable acts like a sentinel. We'll tangle and test this below.

```
const int SIZE {100};
int numbers[SIZE];
```

```

int count = 0;
int num; // this is a sentinel value

// fetch first number
cout << "Enter a number or -1 to quit: ";
cin >> num;
// fetch next numbers (until full or quit)
while (num != -1 && count < SIZE) {
    count++; // increase sentinel value
    numbers[count-1] = num;
    cout << "\nEnter a number or -1 to quit: ";
    cin >> num;
}
// display valid array items
for (int index=0; index < count; index++)
    cout << numbers[index] << " ";

```

- Test it:

```

cd ../src
g++ part.cpp -o part
echo "1 3 4 77 8 33 -1" | ./part

```

```

Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
Enter a number or -1 to quit:
1 3 4 77 8 33

```

23. [Bonus program] Fill array partially with sentinel control

- **This will be a bonus program to be completed at home.**
- **Task:** Write a C++ program that reads an unknown number of items from a file `numbers.txt` into an array `numbers` of `SIZE 100`.
- To read data from a file:

```

#include <fstream>
ifstream inputFile;
inputFile.open("numbers.txt");
// ...
inputFile.close();

```

- **Sample input:** 47 89 65 36 12 25 17 8 62 10 87 62

```

cd ../data
echo "47 89 65 36 12 25 17 8 62 10 87 62" > numbers.txt
cat numbers.txt

```

```

47 89 65 36 12 25 17 8 62 10 87 62

```

- Starter code: tinyurl.com/fill-array-file

```
// Read an unknown number of items from a file `numbers.txt` into
// an array `numbers` fixed at SIZE 100.
// header files
// namespace use
// main pgm
// declarations (array size, array, sentinel, input file)
// open input file
// read input into array (while count < SIZE and EOF not reached)
// close input file
// display numbers read
```

24. Comparing arrays

- Just like you cannot assign an array to another one you can also not compare a whole array to another one.
- What does the following code check and what is the output:

```
int firstArray [] {5,10,15,20,25};
int secondArray [] {5,10,15,20,25};
(firstArray == secondArray) ? (cout<<"same"):(cout<<"different");
```

different

- Here, the third line is a conditional (or **ternary**) operator. Its general form:

```
condition ? expression_if_true : expression_if_false;
```

It is a compact form of:

```
if (condition)
    expression_if_true;
else
    expression_if_false;
```

- Explanation:

```
int firstArray [] {5,10,15,20,25};
int secondArray [] {5,10,15,20,25};
cout << firstArray << endl << secondArray; // compares the addresses of the 1st el
```

0x7ffe535880e0
0x7ffe53588100

- To compare the contents of two arrays, you must compare their elements.

```
const int SIZE {5};
int firstArray [] {5,10,15,20,25};
```

```

int secondArray [] {5,10,15,20,25};
bool arraysEqual = true; // flag variable
int count = 0; // loop counter variable

// Determine whether elements contain same data
while (arraysEqual && count < SIZE) {
    if (firstArray[count] != secondArray[count])
        arraysEqual = false;
    count++;
}
// Print result
cout << boolalpha << "Arrays equal? " << arraysEqual;

```

Arrays equal? true

25. Primer on functions

- Functions ought to be declared with **return type**, **name** and **argument list**. They are then **defined** outside of main, and finally **called** in main.
- Examples:

```

void foo(int x, int y); // no return, takes two integers
int bar(); // returns integer, no arguments
double baz(int [], int); // returns double, takes array

```

- In longer programs, the declaration (or prototype) is bundled with the preprocessor directives (like `#include`) and other global statements, in a header file, "file.h", while the main function is in a file `main.cpp`, and the function definitions go into a file `file.cpp`. All three are linked when the program is compiled.
- Write a C++ program that defines two integer variables, assigns values to them, and calls a function `add` that adds the **two integers** (=arguments) and returns the integer result (=return type) to main and prints it.

```

#include <iostream>
#include <string>
using namespace std;

// function declaration (return integer result) --> add.h
int add(int, int);

int main() // goes into main.cpp
{
    int x, y; // declare and initialize two integer variables
    // function call (pass two integers to be added) & print it
    cout << add(5,12); // stream the result to stdout
    printf("\n%i\n",add(5,12)); // extra layer of formatting
    return 0;
}
// function definition -> add.cpp
int add(int a, int b)
{
    cout << "\nIn function: " << a + b << endl;
    return a + b;
}

```

- The example shows that you can print (cout) from anywhere in the program irrespective of the return type.
- In math: $f(x,y) = x + y$; value for x, value for y, and a rule for f (add)
- In Java, the function could be in a class

```
#include <iostream>
using namespace std;

// function definition
int add(int a, int b)
{
    return a + b;
}

int main()
{
    int x, y; // declare and initialize two integer variables
    // function call (pass two integers to be added) & print it
    cout << add(5,12);
    return 0;
}
```

- An even simpler example with a void function:

```
#include <iostream>
using namespace std;

// function declaration (return integer result)
void hello();

int main()
{
    hello(); // function call (pass two integers to be added)
    return 0;
}

// function definition
void hello()
{
    cout << "Hello";
}
```

26. Arrays as Function Arguments

- To pass an array as an argument to a function, pass the name of the array. The name will decay to a pointer to the first array element.
- Example: showValues of an array

```
#include <iostream>
using namespace std;

void showValues(const int [], int); // prototype

int main()
{
    const int ARRAY_SIZE=8;
    int numbers[ARRAY_SIZE] {5,10,15,20,25,30,35,40};
    showValues(numbers,ARRAY_SIZE); // function call
}
```

```

    return 0;
}
// function definition
void showValues(const int num[], int size)
{
    for (int i=0; i<size; i++)
        cout << num[i] << " ";
}

```

- Would a range-based loop work in showValues, too?

```

void showValues2(int num[])
{
    for (int val : num)
        cout << val;
}

```

- Answer:

The loop inside showValues2 is invalid because it does not know the end of the range. ARRAY_SIZE is not passed on. And even if it was passed on, it would not be used in the loop

- Don't believe me? Let's try it:

```

#include <iostream>
using namespace std;

void showValues2(const int []); // prototype

int main()
{
    const int ARRAY_SIZE=8;
    int numbers[ARRAY_SIZE] {5,10,15,20,25,30,35,40};
    showValues2(numbers); // function call
    return 0;
}
// function definition
void showValues2(int num[])
{
    for (int val : num)
        cout << val;
}

```

- Result (some stuff deleted):

```

In function 'void showValues2(int*)':
23:18: error: 'begin' was not declared in this scope;
did you mean 'std::begin'?
   23 |     for (int val : num)
       |                  ^~~
       |                  std::begin
/usr/include/c++/11/bits/range_access.h:108:37:
note: 'std::begin' declared here
   108 | template<typename _Tp> const _Tp* begin(const valarray<_Tp>&) noexcept;
       |                                     ^~~~~~
23:18: error: 'end' was not declared in this scope; did you mean 'std::end'?
   23 |     for (int val : num)
       |                  ^~~
       |                  std::end

```

```
/usr/include/c++/11/bits/range_access.h:110:37:
note: 'std::end' declared here
template<typename _Tp> const _Tp* end(const valarray<_Tp>&) noexcept;
|                               ^~~
```

27. NEXT Introduction to the STL vector

- The vector data type of the Standard Template Library (STL) is similar to an array:
 1. Holds a sequence of values or elements (of one type).
 2. Stores its elements in contiguous memory locations.
 3. Can be accessed using the subscript operator [] (only on existing vector elements).
- Some differences to arrays:
 1. You don't have to declare the number of elements that the vector will have.
 2. If you add a value to a vector that's already full, the vector will automatically increase its size to accommodate the new value.
 3. A vector can report the number of elements it contains.
 4. To use vector you need using namespace std; in your program (or otherwise declare std::vector).
 5. You need to #include <vector>.
- What the doc says (cppreference.com):

You need to #include <vector> - check out cppreference: "std::vector is a sequence container that encapsulates dynamic size arrays."

Further down the page, you see some notes on the algorithmic complexity (aka efficiency) of common vector operations:

1. Random access (pick any element): $O(1)$
 2. Insertion or removal of elements at the end: $O(1)$
 3. Insertion or removal of elements anywhere: $O(n)$
- Important member functions: swap, clear, empty, push_back, insert, resize, pop_back, at.

28. Declaring vectors

- Example: We define numbers as a vector of integers vector<int>.

```
vector<int> numbers; // Declaring an empty integer vector
vector<int> numbers2(10); // Declaring a starting size is optional.
vector<int> numbers3(10,2); // Initializes 10 numbers to the value 2.
vector<int> numbers4(numbers3); // Copies vector numbers3 into vector numbers4
vector<int> num {10,20,30,40}; // Initializes vector with value set.
```

- Overview of vector definitions for other than int types:

Definition Format	Description
<code>vector<float> amounts;</code>	Defines <code>amounts</code> as an empty vector of floats.
<code>vector<string> names;</code>	Defines <code>names</code> as an empty vector of string objects.
<code>vector<int> scores(15);</code>	Defines <code>scores</code> as a vector of 15 ints.
<code>vector<char> letters(25, 'A');</code>	Defines <code>letters</code> as a vector of 25 characters. Each element is initialized with 'A'.
<code>vector<double> values2(values1);</code>	Defines <code>values2</code> as a vector of doubles. All the elements of <code>values1</code> , which is also a vector of doubles, are copied to <code>value2</code> .

29. **PRACTICE** Storing values in a vector: Payroll program -> CANVAS

- We write a program to compute the `grossPay` of `NUM_EMPLOYEES` employees based on the number of hours each employee has worked, and on the `payRate` each employee receives.
- Sample input: (hours and pay rate for 5 employees)

```
10 9.75 15 8.62 20 10.50 40 18.75 40 15.65
```

- Starter code: tinyurl.com/payroll-cpp
- Solution:

```
// Compute the grossPay of NUM_EMPLOYEES who work certain hours and
// receive a certain payRate.
// Input: integer hours and double payRate for each employee
// Sample output: Hours worked/payRate/grossPay received for each employee
// Author: Marcus Birkenkrahe (based on Gaddis, p.444)
#include <iostream>
// input/output manipulation
// vectors
using namespace std;

int main()
{
    // Declare variables and constants
    // number of employees (5)
    // hours vector
    // payRate vector
    // Loop index
    // Hours x pay rate

    // Ask for user input.
    // Get the hours worked and the hourly payrate for each employee.
```

```

// Print the input after receiving it.

// Display each employee's gross pay (hours x pay rate).
// Set the precision to 2 decimals after the point.

return 0;
}

```

- Solution: <https://onecompiler.com/cpp/43x2syux9>

```

// Compute the grossPay of NUM_EMPLOYEES who work certain hours and
// receive a certain payRate.
// Input: integer hours and double payRate for each employee
// Sample output: Hours worked/payRate/grossPay received for each employee
// Author: Marcus Birkenkrahe (based on Gaddis, p.444)
#include <iostream>
#include <iomanip> // input/output manipulation
#include <vector> // vectors
using namespace std;

int main()
{
    // Declare variables and constants
    const int NUM_EMPLOYEES = 5; // number of employees (5)
    vector<int> hours(NUM_EMPLOYEES); // hours vector
    vector<double> payRate(NUM_EMPLOYEES); // payRate vector
    double grossPay; // gross pay = hours x pay rate

    // Ask for user input.
    cout << "Enter the hours worked by " << NUM_EMPLOYEES
         << " employees and their hourly rates.\n";
    // Get the hours worked and the hourly payrate for each employee.
    // Print the input after receiving it.
    for (int index=0; index < NUM_EMPLOYEES; index++)
    {
        cout << "Hours worked by employee #" << (index + 1)
             << ": ";
        cin >> hours[index];
        cout << hours[index] << endl;
        cout << "Hourly pay for employee #" << (index + 1)
             << ": ";
        cin >> payRate[index];
        cout << payRate[index] << endl;
    }

    // Display each employee's gross pay (hours x pay rate).
    // Set the precision to 2 decimals after the point.
    cout << "\nHere is the gross pay for each employee:\n";
    cout << fixed << showpoint << setprecision(2);
    for (int index=0; index < NUM_EMPLOYEES; index++)
    {
        grossPay = hours[index] * payRate[index];
        cout << "Employee #" << (index + 1)
             << ": " << grossPay << endl;
    }
    return 0;
}

```

Enter the hours worked by 5 employees and their hourly rates.
Hours worked by employee #1: 0
Hourly pay for employee #1: 0

```
Hours worked by employee #2: 0
Hourly pay for employee #2: 0
Hours worked by employee #3: 0
Hourly pay for employee #3: 0
Hours worked by employee #4: 0
Hourly pay for employee #4: 0
Hours worked by employee #5: 0
Hourly pay for employee #5: 0
```

Here is the gross pay for each employee:

```
Employee #1: 0.00
Employee #2: 0.00
Employee #3: 0.00
Employee #4: 0.00
Employee #5: 0.00
```

- Compile and run it with the sample input:

```
cd ../src
make payroll2
echo "10 9.75 15 8.62 20 10.50 40 18.75 40 15.65" | ./payroll2
```

```
make: 'payroll2' is up to date.
Enter the hours worked by 5 employees and their hourly rates.
```

```
Hours worked by employee #1: 10
Hourly pay for employee #1: 9.75
Hours worked by employee #2: 15
Hourly pay for employee #2: 8.62
Hours worked by employee #3: 20
Hourly pay for employee #3: 10.5
Hours worked by employee #4: 40
Hourly pay for employee #4: 18.75
Hours worked by employee #5: 40
Hourly pay for employee #5: 15.65
Here is the gross pay for each employee:
Employee #1: 97.50
Employee #2: 129.30
Employee #3: 210.00
Employee #4: 750.00
Employee #5: 626.00
```

30. Scope

- When is there a semi-colon? Only for ordinary statements, not for control statements with their own {...} scope, or function definitions. **Scope** defines where a variable, parameter or function is **visible** and **usable**.

```
for (int i=0; i<SIZE;i++) // control statement
{
    // loop body
    i * i;
    cout << i; // statement
}

if(vector.empty())
{
    // if statement body
    cout << "empty";
}
```

```

}

void ask(int); // prototype

int ask(int n)
{
    // function body
    return n * n;
}

```

31. Using the range-based for loop with a vector (from C++11)

- Example 1:

```

// Display vector elements using a range-base for loop
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define and initialize a vector.
    vector<int> numbers {10,20,30,40,50};
    // Display the vector elements.
    for (int val : numbers)
        cout << val << " ";

    return 0;
}

```

10 20 30 40 50

- Example 2:

```

// Using a range-based for loop, fill a vector with user input
// and display the elements of the vector
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define a vector.
    vector<int> numbers(5); // starting size 5
    // Get values for the vector elements.
    for(int &val : numbers)
        cin >> val;
    // Display the vector elements.
    for(int val : numbers)
        cout << val << " ";

    return 0;
}

```

0 0 0 0 0

- Input file:

```
cd ../data
echo "10 20 30 40 50" > inputFile
cat inputFile
```

```
10 20 30 40 50
```

32. **PRACTICE** Using the `push_back` member function (payroll pgm modified) -> CANVAS

- You use `push_back` to store a value in a vector without starting size, or to append it at the end of a vector that is already full.
- Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers;    // empty vector, no starting size

    // print vector elements
    for (int n : numbers)
        cout << n << " ";

    // add some values using push_back
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    // print vector elements
    for (int n : numbers)
        cout << n << " ";

    return 0;
}
```

```
10 20 30
```

- Let's modify the payroll program: Change the vector definitions to vectors without starting size, and use `push_back` to fill the vectors.
- Sample input: number of employees | hours | pay rate for each

```
5 10 9.75 15 8.62 20 10.50 40 18.75 40 15.65
```

- Starter code: tinyurl.com/payroll-2-cpp

```

// Program stores, in two vectors, the hours worked by the employees
// and their hourly pay rates, and computes and displays the gross pay
// for each employee.
// header files

// main program

// Declare variables: hours, pay rate, number of employees

// Get the number of employees.

// Input the data.

// Display each employee's gross pay (hours x pay rate)

```

- Solution: <https://onecompiler.com/cpp/43x2uk9xg>

```

// Program stores, in two vectors, the hours worked by the employees
// and their hourly pay rates, and computes and displays the gross pay
// for each employee. Here, numEmployees is an integer variable;
// Sample input: 5 10 9.75 15 8.62 20 10.50 40 18.75 40 15.65

// header files
#include <iostream>
#include <iomanip>
#include <vector>

// main program
int main()
{
    // Declare variables: hours, pay rate, number of employees
    std::vector<int> hours;
    std::vector<double> payRate;
    int numEmployees;
    double grossPay; // to be computed
    int tempHours;    // stores temporary hours input
    double tempRate;  // stores temporary pay rate input
    // Get the number of employees.
    std::cout << "How many employees do you have to pay?\n";
    std::cin >> numEmployees;

    // Input the data.
    for(int index=0; index<numEmployees; index++)
    {
        std::cin >> tempHours; // get hours for employee index + 1
        hours.push_back(tempHours); // fill the hours vector
        std::cin >> tempRate; // get pay rate for employee index + 1
        payRate.push_back(tempRate); // fill the payRate vector
    }

    // Display each employee's gross pay (hours x pay rate)
    std::cout << "\nHere is the gross pay for each employee:\n";
    std::cout << std::fixed << std::showpoint << std::setprecision(2);
    for(int index=0; index<numEmployees; index++)
    {
        grossPay = hours[index] * payRate[index];
        std::cout << "Employee #" << (index + 1)
                    << ": $" << grossPay << std::endl;
    }

    return 0;
}

```

How many employees do you have to pay?

Here is the gross pay for each employee:

- Compile and run it with the sample input:

```
cd ../src
make payroll3
echo "5 9.75 15 8.62 20 10.50 40 18.75 40 15.65" | ./payroll3
```

```
make: 'payroll3' is up to date.
How many employees do you have to pay?

Here is the gross pay for each employee:
Employee #1: 6.75
Employee #2: 129.30
Employee #3: 210.00
Employee #4: 750.00
Employee #5: 626.00
```

- Why can't you use `.push_back` directly in a `cout << stream?`

`push_back` returns `void` (a non-value), and the `<<` operator expects a value.

33. Determining the size of a vector

- Code along with me. Starter code: tinyurl.com/vector-size
- Example: Demonstrates the vector method `size`. Notice that the function arguments does not need to include the size since vector is dynamically allocated. Expected output: 0 2 4 6 8 10 12

```
// Demonstrate the size() method of the vector class.
// Author: Marcus Birkenkrahe (pledged) [modified from Gaddis, 2024]
#include <iostream>
#include <vector>
using namespace std;

// Function prototype

int main()
{
    // Define a vector.
    vector<int> numbers;
    // Put a series of numbers in the vector.
    for (int i=0; i<=12; i+=2)
    {
        numbers.push_back(i);
    }
    // Display the numbers.
    showValues(numbers);

    return 0;
}

//*****
// showValues accepts an integer vector as its argument. *
```

```
// The value of each of the vector's elements is displayed. *
//*****
void showValues(vector<int> vect)
{
    for (int i = 0; i < vect.size(); i++)
        cout << vect[i] << " ";
}
```

34. Removing elements from a vector with pop.back

- Code along with me. Starter code: tinyurl.com/vector-pop-back
- .pop_back() removes the last element from the vector.
- Example:

```
// Demonstrate the popback method of the vector class.
// Author: Marcus Birkenkrahe (pledged) [modified from Gaddis, 2024]
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // Define an empty vector
    vector<int> values;

    // Store values in the vector, then display size.
    values.push_back(1);
    values.push_back(2);
    values.push_back(3);

    cout << "The size of values is " << values.size() << endl;

    cout << "Let's pop the values back from the vector:\n";
    // Remove a value from the vector, then display size.
    values.pop_back();
    cout << "The size of values is " << values.size() << endl;
    // Remove a value from the vector then display size.
    values.pop_back();
    cout << "The size of values is " << values.size() << endl;
    // Remove a value from the vector then display size.
    values.pop_back();
    cout << "The size of values is " << values.size() << endl;

    // Removing one more element causes undefined behavior!
    //values.pop_back();
    //cout << "The size of values is " << values.size() << endl;

    return 0;
}
```

```
The size of values is 3
Let's pop the values back from the vector:
The size of values is 2
The size of values is 1
The size of values is 0
```

- What if we were to try and remove another value from the empty vector?

Causes undefined behavior! Not good! Bad! Let's show that.

35. Clearing a vector

- Code along with me. Starter code: tinyurl.com/vector-clear
- To clear the contents of a vector, use the `.clear()` member function.
- Example:

```
// Demonstrate the clear() method of the vector class.
// Author: Marcus Birkenkrahe (pledged) [modified from Gaddis, 2024]
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    // define an empty vector of 100 integer elements
    vector<int> values(100);

    // display vector size
    cout << "The vector has "
         << values.size() << " elements.\n";

    // clear vector
    values.clear();

    // display vector size
    cout << "The vector now has "
         << values.size() << " elements.\n";

    return 0;
}
```

```
The vector has 100 elements.
The vector now has 0 elements.
```

36. Detecting an empty vector

- Code along with me. Starter code: tinyurl.com/vector-empty
- To determine if a vector has no content (that is no memory reserved for it), use the `.empty` member function.
- Example: To demonstrate the use of the `empty` function, we store values in a vector, and then determine the average of these values. If the vector is not empty, there are no values to average and we set the average to zero.

```
// Demonstrate the empty() method of the vector class.
// Get the number of integer values from the user, and the values,
// and then average over them and print the average
// Author: Marcus Birkenkrahe (pledged) [modified from Gaddis, 2024]
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;
```

```

// Function prototype
// Return a double and takes an integer vector as argument
double avgVector(vector<int>);

int main()
{
    // define empty integer vector, number of values, and average
    vector<int> values; // holds the input values
    int numValues; // number of values to average
    double average; // store the average of the values

    // Get the number of values to average.
    cout << "How many values do you wish to average? ";
    cin >> numValues;

    // Get the values and store them in the vector.
    for (int count=0; count < numValues; count++)
    {
        int tempValue;
        cout << "Enter an integer value: ";
        cin >> tempValue;
        values.push_back(tempValue);
    }
    // Get the average of the values and display it.
    average = avgVector(values);
    cout << fixed << showpoint << setprecision(2);
    cout << "\nAverage: " << average << endl;

    return 0;
}

//*****
// avgVector accepts an integer vector as its argument *
// If the vector contains values, the function returns *
// the average of those values as a double. Otherwise, *
// an error message is displayed and 0.0 is returned. *
//*****
double avgVector(vector<int> vect)
{
    double total = 0.;
    double avg;
    if (vect.empty()) // check if vector is empty
    {
        cout << "\nVector is empty - nothing to average.\n";
        avg = 0.0;
    }
    else
    {
        for (int count=0; count<vect.size(); count++)
            total += vect[count];
        avg = total / vect.size();
    }
    return avg;
}

```

How many values do you wish to average? Enter an integer value: Enter an integer v
Average: 44.00

- InputFile:

```
cd ../data
echo "5 12 18 3 7 9" > inputFile
#echo "0" > inputFile
cat inputFile
```

```
5 12 18 3 7 9
```

37. **PRACTICE** Summary of vector member functions

- Open a new C++ file and code along with me. For each new member function, delete the old code. You can find the solutions in GitHub.
- `at(element)` returns the value of the element located at `element` in the vector:

```
vector<int> vect(100,10);
cout << vect.at(99); // going outside of range triggers an exception
```

```
10
```

- `resize(num, value)` resizes a vector by `num` elements and initializes each of the new elements with `value`. If `num` is the same as the current size, it does nothing.

```
vector<int> vect {1,2,3,4,5};
cout << vect.size() << endl;
vect.resize(10,100);
cout << vect.size() << endl;
cout << vect[9];
```

```
5
10
100
```

- `swap(vector2)` swaps the content of the vector with the contents of `vector2`.

```
vector<int> vect1 {10,20,30};
vector<int> vect2 {40,50,60};

vect1.swap(vect2);

for (int val : vect1)
    cout << val << " ";

cout << endl;

for (int val : vect2)
    cout << val << " ";
```

```
40 50 60
10 20 30
```

- `clear()` clears a vector of all its elements:

```
vector<string> names {"Joe", "Jim", "Jane"};
for (std::string name : names) cout << name << " ";
names.clear();
for (std::string name : names) cout << name << " - "; // notice that NOTHING is pr
```

Joe Jim Jane

- `empty()` returns true if the vector is empty, and false otherwise.

```
vector<bool> Boolean {0,1,1,1,0};
vector<bool> Boolean2 {};

cout << "\nVector is: ";
Boolean.empty() ? cout << "empty" : cout << "not empty";
cout << "\nVector is: ";
Boolean2.empty() ? cout << "empty" : cout << "not empty";
```

Vector is: not empty
Vector is: empty

- Challenge: Simplify this last program by moving the `empty()` check into a function `isEmpty`.
- Solution:

```
// pgm checks if vector is empty

#include <iostream>
#include <vector>

void isEmpty(std::vector<bool>);

int main()
{
    std::vector<bool> Boolean {0,1,1,1,0};
    std::vector<bool> Boolean2 {};

    isEmpty(Boolean);
    isEmpty(Boolean2);

    return 0;
}

void isEmpty(std::vector<bool> vec)
{
    std::cout << "\nVector is ";
    if (vec.empty())
    {
        std::cout << "empty";
    } else {
        std::cout << "not empty";
    }
}
```

```
Vector is not empty  
Vector is empty
```

- `pop_back` removes the last element from the vector thus reducing its size by one.

```
vector<double> pi {14, 1459, 145926};  
for (double digit : pi) cout << "3." << digit << endl;  
pi.pop_back();  
for (double digit : pi) cout << "3." << digit << endl;
```

```
3.14  
3.1459  
3.145926  
3.14  
3.1459
```

- `push_back(value)` stores value in the last element of the vector. If the vector is full or empty, a new element is created.

Challenge: Write a void function `append` that you can use to `push_back` the value 1 to a vector `id`. Call the function three times on an empty vector, then print the result. To work on the original vector, make sure that you pass a reference, and not a copy of the vector. **Do not use** using namespace `std`;

Solution:

```
#include <iostream>  
#include <vector>  
  
void append(std::vector<int> &);  
  
int main()  
{  
    std::vector<int> id;  
    append(id);  
    for (int elem : id) std::cout << elem << " ";  
    return 0;  
}  
  
void append(std::vector<int> &vec)  
{  
    vec.push_back(1);  
}
```

```
1
```